Автономная некоммерческая организация высшего образования
«Университет Иннополис»
(АНО ВО «Университет Иннополис»)

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(БАКАЛАВРСКАЯ РАБОТА)
по направлению подготовки
09.03.01 – «Информатика и вычислительная техника»

GRADUATION THESIS
(BACHELOR'S GRADUATION THESIS)
Field of Study
09.03.01 – «Computer Science»

Направленность (профиль) образовательной программы
«Информатика и вычислительная техника»
Area of Specialization / Academic Program Title:
«Computer Science»

Тема / Topic	**Разработка и оценка интеллектуальной системы для подготовки отчётов о программных ошибках / Development and Evaluation of an Intelligent System for Software Bug Reporting**

Работу выполнил / Thesis is executed by	**Андуков Алмаз Алексеевич / Andukov Almaz Alexeevich**	подпись / signature
Руководитель выпускной квалификационной работы / Graduation Thesis Supervisor	**Махмуд Надери / Mahmoud Naderi**	подпись / signature
Консультанты / Consultants		подпись / signature

Иннополис, Innopolis, 2026

Contents

1 Introduction **7**

1.1 Background and Motivation 7

1.2 Problem Statement 8

1.3 Aim and Objectives 10

1.3.1 Research Questions 11

1.4 Scope and Expected Contribution 12

1.5 Thesis Structure 14

2 Literature Review **15**

2.1 The Role of Bug Reports in Software Maintenance 15

2.2 Completeness, Clarity, and Factual Accuracy 17

2.3 Structured Forms and Interactive Reporting 18

2.4 Natural Language Processing and Large Language Models . . . 19

2.5 LLM-Based Bug Report Improvement 21

2.6 Service Documentation and MCP 22

2.7 Evaluation Practices in Prior Research 24

2.8 Synthesis, Research Gap, and Research Questions 25

Bibliography cited **28**

Abstract

Bug reports submitted through technical support channels may omit reproduction steps, expected behavior, or the context in which a problem occurred. Collecting this information requires clarification between customers and technical teams. Intelligent reporting assistance offers a way to obtain more precise descriptions during report preparation, but its value must be assessed alongside the effort required from the reporter and the accuracy of the resulting information.

This thesis aims to develop and evaluate an intelligent system for software bug reporting that companies can integrate into their service support processes. The proposed system will use a large language model (LLM) to identify missing or ambiguous information and ask service users targeted clarification questions. Through the Model Context Protocol (MCP), it will access documentation describing the service's capabilities and previously resolved problems to inform those questions. The system is designed to interact with users in their chosen language and organize their confirmed answers into a structured report for developers.

The planned evaluation will compare the system with a conventional structured form and with the same assisted form without documentation access. It will examine report completeness and clarity, reporting time, interaction effort, perceived workload, and the preservation of user-confirmed information. The documentation comparison will assess its additional contribution to question relevance and report completeness. The intended contribution is a prototype and empirical evidence about the benefits and limitations of documentation-informed assistance for preparing accurate and useful bug reports.

Chapter 1

Introduction

1.1 Background and Motivation

Bug reports provide developers with information about problems encountered during software use. Their usefulness depends on whether the reported behavior and the circumstances in which it occurred can be understood. Zimmermann et al. identified a mismatch between the information developers consider useful and the information reporters can provide: reproduction steps, stack traces, and test cases were valued by developers but could be difficult for reporters to supply [1]. Soltani et al. subsequently examined report elements through interviews, a developer survey, and repository analysis, reinforcing the importance of reproduction steps and contextual information [2].

Missing information is not limited to diagnostic artifacts. Chaparro et al. found that expected behavior and steps to reproduce were often absent from bug descriptions [3]. In a study of web application reports, Wang et al. found that usage information was frequently added after the initial submission [4]. These findings motivate assistance that helps reporters describe their observations and relevant circumstances before submitting a report. They do not imply that every report needs the same technical details.

Interactive assistance is an established direction in bug reporting. BURT guides end-users through reporting observed behavior, expected behavior, and reproduction steps, using immediate feedback and application-specific information [5]. More recent approaches investigate LLM-based report improvement and external context: ChatBR assesses and improves report elements, while ImproBR combines report improvement with retrieved domain knowledge [6], [7]. The present thesis builds on these directions by investigating documentation-informed clarification in an interaction with the reporter.

The proposed system will use a large language model (LLM) to identify omissions or ambiguity and formulate follow-up questions. Documentation accessed through the Model Context Protocol (MCP) will provide information about the service's capabilities and previously resolved problems. MCP defines mechanisms for exposing resources and callable tools; deciding which documents are relevant and how to use them remains a responsibility of the reporting system [8], [9].

The motivation is to examine whether this combination helps service users provide more complete and understandable reports while preserving their observations. This requires assessing both the resulting information and the effort involved in supplying it. A longer or more fluent report is not sufficient evidence of success: generated details can be unsupported by the source information, as discussed in research on hallucination in natural-language generation [10].

1.2 Problem Statement

The problem addressed in this thesis is the collection of sufficiently complete, clear, and accurate information from users reporting problems with a company's software services. A report may describe a symptom without specifying the actions that preceded it, the expected outcome, or relevant preconditions. Ex-

isting research establishes the occurrence of such omissions and the importance of additional context [3], [4]. The research problem is to determine whether an intelligent reporting interaction can address these omissions effectively in the selected support setting.

The intended organizational clients are companies that can integrate the reporting system into support processes for their own software services. The system is conceived as a component that can be adapted to a company's services and documentation, without requiring a particular software delivery model. The project is an independently developed research prototype with no industrial partner. Its evaluation will use controlled support scenarios rather than assume an existing industrial deployment.

The organizations adopting the system and the people submitting reports have different roles. Reporters are users of the adopting company's services, while developers and support specialists receive the structured reports. The reporting interaction should accommodate users with different levels of technical knowledge. It does not require them to identify the cause of a problem or have access to source code and internal diagnostic tools.

The intended intake covers user-observed problems such as an unresponsive button, an incorrectly displayed interface, an error during an operation, or uncertainty about how to use a service. The last category includes cases in which a user does not know that a prerequisite or an earlier action is required. These are examples of the supported reporting context, not an exhaustive list of defects or a claim that all problem types have been evaluated. An incoming support request is not necessarily a confirmed software bug: it may describe a defect, unexpected behavior, or a misunderstanding of the documented workflow. The system's task is to clarify and record the problem so that the technical team has an adequate description for further assessment.

For each adopting company, the planned documentation collection will de-

scribe its service's capabilities and previously resolved problems. The system will access this collection through MCP and use relevant material to ask questions about the operation attempted, its inputs, and applicable conditions. For example, a documented prerequisite may motivate a question about whether the reporter completed the required action. Documentation about another incident can suggest what to clarify, but it does not establish what happened in the current case. The final report will organize information supplied or confirmed by the reporter and preserve unresolved details as such.

The interface is intended to accept users' descriptions and provide clarification questions in their chosen language. It will then produce a structured report for developers. The languages evaluated and the specific service workflows and documentation used will be defined in the design and evaluation chapters. This requirement provides a direction for implementation; it does not establish equivalent reporting quality across all languages.

The central uncertainty is whether adaptive questions informed by documentation improve reporting outcomes relative to a conventional structured form. Additional questions may collect useful information while also increasing reporting time or workload. A further uncertainty concerns the contribution of documentation itself: a comparison with an assisted form without documentation is needed to distinguish that contribution from the effect of the interaction. The thesis therefore treats usefulness as an empirical question involving report quality, factual fidelity, and user effort.

1.3 Aim and Objectives

The aim of this thesis is to develop an intelligent system that companies can integrate into their software support processes and evaluate how LLM-assisted clarification, informed by service documentation accessed through MCP, affects

report quality and reporting effort in controlled support scenarios.

The following objectives support this aim:

1. Review research on bug report quality, interactive reporting, LLM-based report improvement, and the use of external context.

2. Define representative reporting scenarios, relevant report elements, and a documentation collection covering service capabilities and previously resolved problems.

3. Design and implement a reporting prototype that detects missing or ambiguous information, asks targeted clarification questions, and structures the reporter's confirmed answers.

4. Integrate access to service documentation through MCP and use retrieved information to inform clarification questions.

5. Prepare a conventional structured form and an assisted condition without documentation access to support comparative evaluation.

6. Evaluate report completeness, clarity, and fidelity to user-provided information, together with reporting time, interaction effort, perceived workload, and question relevance.

7. Analyze the results to identify the system's benefits, limitations, and the additional contribution of documentation access.

1.3.1 Research Questions

The objectives concern building and studying the system. The following research questions specify the outcomes that the evaluation will investigate:

RQ1: How does an LLM-assisted reporting form affect the completeness and clarity of bug reports compared with a conventional structured form?

RQ2: How does the form affect reporting time, interaction effort, and perceived workload?

RQ3: How faithfully do the final reports preserve user-confirmed information, and what unsupported details, if any, are introduced?

RQ4: What additional effect does access to service documentation through MCP have on the relevance of clarification questions and the completeness of the resulting reports, compared with the same assisted form without documentation access?

These questions allow improvements, trade-offs, or a lack of measurable benefit. RQ1 and RQ2 address report quality and reporting effort, respectively. RQ3 examines whether the output remains faithful to the reporter's evidence. RQ4 isolates the additional contribution of documentation. Chapter 2 explains how the reviewed literature motivates these questions; the evaluation chapter will define their measurements and analysis.

1.4 Scope and Expected Contribution

The scope covers the preparation of reports about user-observed software problems, from an initial description through clarification to a structured report for technical assessment. The intake includes apparent functional failures, interface problems, execution errors, and difficulties understanding how to use a service. Reports about suspected defects are the focus of the bug reporting research questions; requests arising from unclear usage or prerequisites provide a related support context and will be distinguished in the evaluation. Collecting such requests does not imply that the system will automatically determine whether a software defect exists.

The prototype will combine a user-facing reporting interface, LLM-assisted information collection, and documentation access through MCP. Integration is intended to connect the reporting process to the adopting company's documentation. The service documentation and resolved cases used in the study will be identified so that their provenance and relevance can be assessed. Adaptability to different companies is a design objective; the evaluation will establish results only for the workflows and materials actually studied.

The study focuses on what the reporting interaction collects and communicates. Report completeness concerns relevant information rather than the completion of every possible field. Clarity concerns whether the description is understandable, while fidelity concerns whether observations and confirmed answers are preserved without unsupported additions. The evaluation will consider these properties alongside the effort required from the reporter and the relevance of the system's questions.

Root-cause identification, automated code repair, and deployment of fixes are outside the scope. A more complete problem description may support later investigation, but improvements in reporting will not be presented as evidence of faster diagnosis or resolution. Such claims would require measurements of those downstream activities. Research on non-reproducible reports also shows that reproduction difficulties can arise from environmental differences and dependencies between bugs, beyond the wording of a report [11].

The expected contribution is a working prototype and a comparative evaluation of documentation-informed reporting assistance. The contribution lies in implementing and assessing the interaction in the defined setting, including its factual fidelity and the additional value of documentation. It does not depend on claiming that interactive reporting or the use of external knowledge is new. The conclusions will be bounded by the service scenario, reporting tasks, languages, documentation, and participants included in the study.

1.5 Thesis Structure

The thesis is organized as follows. Chapter 2 reviews bug report quality, structured and interactive reporting, LLM-assisted report improvement, documentation access through MCP, and evaluation practices. It synthesizes the literature in relation to the research questions introduced in this chapter. Chapter 3 will describe the service scenario, requirements, documentation sources, and system architecture. Chapter 4 will present the implementation, including the reporting interaction, documentation integration, and generation of structured reports. Chapter 5 will specify the evaluation methodology and report and discuss the results, including limitations and threats to validity. Chapter 6 will summarize the findings and outline directions for further work.

Chapter 2

Literature Review

This chapter presents a focused narrative review of research on bug report quality, interactive reporting, and language-model-assisted report improvement. It also examines the use of external documentation and the Model Context Protocol (MCP), which are relevant to the proposed system. The review distinguishes evidence about report content from evidence about the reporting interaction and subsequent debugging. It does not claim exhaustive coverage or constitute a systematic literature review.

2.1 The Role of Bug Reports in Software Maintenance

Bug reports connect a user's account of a software problem with the information needed by developers to investigate it. Zimmermann et al. surveyed developers and users of Apache, Eclipse, and Mozilla and identified a mismatch between the information developers need and what reporters can provide. Reproduction steps, stack traces, and test cases were considered useful but were also difficult for reporters to supply [1]. This finding motivates reporting assistance that helps people communicate evidence they possess, while recognizing that they may not

have access to diagnostic artifacts.

Soltani et al. examined the importance of report elements through interviews with 35 developers, a survey of 305 developers, and an analysis of issue repositories from 250 projects. Their results emphasize reproduction steps and stack traces, while also considering software versions, user content, and other contextual information [2]. The study provides broader evidence that useful report content extends beyond a general symptom description. Its repository analysis concerns associations between report elements and resolution times; such associations do not establish that adding a field to a reporting interface will cause faster resolution.

Wang et al. compared initial and final versions of 1,000 bug reports from ten open-source web applications. They found that end-user usage information and developers' diagnostic suggestions were often absent initially and added during later discussions [4]. This report evolution suggests an opportunity to collect relevant usage context earlier. However, diagnostic suggestions added during investigation should not be confused with facts that the original reporter could have supplied.

Erfani Joorabchi et al. studied non-reproducible reports in one industrial and five open-source repositories. Their analysis identified several reasons for non-reproducibility, including environmental differences, insufficient information, and dependencies between bugs [11]. Consequently, a clear report can support investigation without ensuring successful reproduction: some obstacles concern the software and its environment rather than the quality of the description.

Together, these studies justify improving information collection at reporting time. They also limit the conclusions that can be drawn from report quality alone. A reporting system can improve the evidence available for investigation, but its effect on defect localization or repair must be demonstrated separately.

2.2 Completeness, Clarity, and Factual Accuracy

Chaparro et al. analyzed 2,912 reports and investigated the detection of missing expected behavior and steps to reproduce. Their findings show that these elements are frequently absent, even when observed behavior is described [3]. This distinction is important because describing a failure does not necessarily explain either the intended outcome or the actions that preceded it.

The following information categories synthesize the report elements investigated in the literature [1], [2], [3], [4]:

- **Observed behavior:** what happened, including the visible symptom or available error message.
- **Expected behavior:** the outcome the reporter expected, with its basis clarified where necessary.
- **Steps to reproduce:** the actions, inputs, and relevant preconditions preceding the problem.
- **Context:** applicable details about the service, version, environment, and circumstances of use.
- **Supporting evidence:** screenshots, logs, stack traces, or other artifacts when available and relevant.

Completeness is only one aspect of quality. Chaparro et al.'s EULER approach assesses reproduction steps by relating natural-language descriptions to application interactions and producing feedback about problematic or missing steps [12]. A report can therefore contain a reproduction section while still omitting necessary actions or describing them ambiguously. Counting completed fields alone cannot establish whether another person can follow the report.

Factual accuracy creates a further distinction for generated reports. Ji et al. describe hallucination in natural-language generation as output that is unsupported by, or inconsistent with, the relevant source information [10]. Applied to reporting, a fluent description may still contain an invented error message, an unconfirmed action, or an altered observation. These examples are consequences of the distinction between linguistic plausibility and evidential support, rather than findings about the proposed prototype.

For this thesis, completeness refers to the presence of relevant information, clarity to its comprehensibility, and factual accuracy to its consistency with available evidence. A field that is inapplicable or genuinely unknown should remain distinguishable from an omission. These are analytical definitions used to organize the review; their operational assessment belongs to the evaluation methodology.

2.3 Structured Forms and Interactive Reporting

Structured templates organize information into predefined categories. The template-based reporting interface used as a comparison in the BURT study illustrates this approach: reporters supply information through designated sections rather than receiving an adaptive dialogue [5]. Structure supplies an organizational framework, but the effectiveness of additional guidance remains an empirical question.

CUEZILLA, described by Zimmermann et al., assesses report quality and recommends additional report elements. It was trained and evaluated using 289 reports rated by developers [1]. It establishes that feedback during reporting predates LLM-based systems. In this review, the journal article provides the principal account of that work; the earlier conference publication is not treated as an independent replication.

Song et al. introduced BURT, a task-oriented chatbot that guides end-users through observed behavior, expected behavior, and reproduction steps. It provides immediate quality feedback, clarification prompts, and graphical suggestions. Its implemented reporting interface is web-based, while its application analysis and evaluation concern Android apps [5]. These are different platform properties: a browser-based interface does not make the evaluated reporting technique a method for arbitrary web applications.

BURT combines language processing with a graph of application screens and interactions derived from execution data. Its evaluation used twelve bugs from six Android apps, compared reports with those from a template-based system, and examined users' experiences and the correctness of reported information [5]. The results support interactive guidance in the studied setting. They do not establish equivalent effects for every service or documentation source. Importantly, interaction and usability are already part of this line of work, so considering user effort is not by itself a new contribution.

The relevant distinction is therefore between different mechanisms for guidance and different sources of application context. The proposed system will use service documentation accessed through MCP to inform clarification questions. Whether this approach produces useful questions and preserves the reporter's account requires evaluation; the existence of an interactive interface alone does not distinguish it from BURT.

2.4 Natural Language Processing and Large Language Models

The missing-information detection approach of Chaparro et al. demonstrates that bug descriptions can be analyzed automatically for particular report elements [3]. Detection, however, answers a narrower question than conversational assis-

tance. Recognizing an absent reproduction description does not directly establish which question should be asked next or which service-specific precondition matters.

Hou et al.'s systematic literature review describes applications of LLMs across software engineering and discusses challenges in their use and evaluation [13]. For reporting, language models offer a way to interpret varied descriptions and produce questions or summaries. This is a rationale for investigating them, rather than evidence that an LLM necessarily outperforms a specialized classifier or a carefully designed form.

McGowan et al. describe work on using application interface and execution information to prompt an LLM to improve reports and predict reproduction steps [14]. Their publication states that evaluation of the tool was ongoing. It therefore provides an example of an application-informed design, but should not be cited as a completed demonstration of improved reporting performance.

Hallucination is a material limitation when generation is used to complete an account of an actual event [10]. In the proposed system, a question suggested by documentation will be treated as a request for confirmation. A documented upload limit, for example, can motivate asking about file size; it cannot establish the size of the file the user actually submitted. This example illustrates the intended separation between documented behavior, user observations, and unresolved information.

The role investigated here is assistance with problem description and information collection. Identifying what the user observed and under which conditions is different from determining the underlying code defect. The literature on report analysis provides a foundation for the former, without removing the need for independent evidence about the latter.

2.5 LLM-Based Bug Report Improvement

Bo et al. introduced ChatBR, which combines report-element assessment with ChatGPT-based improvement. The approach addresses observed behavior, expected behavior, and reproduction steps, and evaluates missing-information detection and generated content [6]. This is relevant to identifying deficiencies and producing structured reports. However, similarity to a reference report is a different outcome from obtaining confirmation from the person who experienced a particular failure.

Acharya and Ginde evaluate instruction-tuned models and ChatGPT for converting informal reports into structured reports. Their study uses quality and text-similarity measures and compares performance across models and report elements [15]. The published EASE 2025 paper is used here rather than its earlier arXiv version. Its results concern report generation under the studied conditions; they do not directly measure an interactive reporter's workload.

Mahmud et al. introduced AstroBR, which uses an LLM together with a dynamically constructed application model to identify, extract, and assess reproduction steps [16]. Its evaluation compares step annotations and missing-step suggestions with reference annotations and EULER. The approach demonstrates that LLM-based reporting assistance can use application-specific evidence beyond the report text. Its reliance on GUI execution models distinguishes its context source from the service documentation proposed here, without implying that either source is universally superior.

Mukherjee and Rahman's BugMentor retrieves relevant historical reports and uses neural text generation to answer follow-up questions about an existing report. The evaluation includes reference-answer metrics and a developer study [17]. BugMentor thus addresses an important part of the clarification process, but generating an answer from historical context differs from asking the current reporter to supply an observation. This distinction concerns the origin of the in-

formation, not simply whether retrieval is used.

Akyol et al.'s ImproBR combines detection of incomplete or ambiguous report sections with LLM-based improvement and retrieval from Minecraft Wiki. It is listed in the EASE 2026 research programme; the accessible author manuscript is cited explicitly [7]. Its evaluation includes structural completeness, executability, and reproducibility of Minecraft reports. This is a particularly close example of report improvement using external domain knowledge. Accordingly, combining bug reporting with retrieved documentation cannot itself be claimed as an unexplored idea. The present thesis instead investigates documentation-informed questions within an interaction with the reporter.

Chen et al. investigate decomposing complex bug reports using LLMs and show that performance depends on prompting [18]. Their arXiv manuscript is cited as a preprint. Although decomposition differs from information collection, it provides a related example of the need to evaluate the interpretation of report content rather than assuming correct understanding.

Across these approaches, three operations should be distinguished: detecting a deficiency, generating a candidate completion, and collecting confirmed information. The proposed system focuses on the third operation while using the first to guide the interaction. It may structure the answers, but any proposed completion remains subject to confirmation.

2.6 Service Documentation and MCP

Lewis et al. introduced retrieval-augmented generation (RAG), combining a generative model with information retrieved from an external collection. Their experiments concern knowledge-intensive NLP tasks and show benefits of retrieval in those settings [19]. The general relevance to this thesis is the use of identifiable external information rather than reliance on model parameters alone. Those

experiments do not directly establish the quality of documentation-informed bug-reporting questions.

External context is already represented in the reviewed reporting literature: AstroBR uses an application execution model, BugMentor retrieves historical reports, and ImproBR retrieves domain knowledge [7], [16], [17]. These sources answer different questions. Documentation can describe supported operations and prerequisites, while an execution trace or the reporter's answer can supply evidence about what actually happened.

MCP standardizes communication between applications and servers exposing context and capabilities. Its specification defines resources for application-managed contextual data and tools for callable operations [8], [9]. A documentation integration may expose documents as resources or provide search and retrieval tools. MCP does not itself choose the relevant passages, establish their correctness, or guarantee that a model's output is grounded. It is the access mechanism; retrieval and question-generation policies remain separate system responsibilities.

The intended role of MCP in this thesis is to provide access to documentation of the services about which users report problems. The system will use that material to formulate questions about relevant operations, inputs, and preconditions, with the purpose of making the reported problem more precisely specified. This is a design commitment, not a result established by the reviewed literature. It does not require assuming a particular service, implementation stack, or industrial partner before those details are defined.

Access to more text also does not ensure effective use of that text. Liu et al. found that language-model performance on long-context tasks can depend on the position of relevant information, with poorer performance when it appears in the middle [20]. This finding motivates selecting relevant documentation rather than treating a large context window as sufficient. Together with the hallucination

literature, it supports assessing whether the questions and summaries are actually supported by the retrieved material [10].

For the proposed reporting interaction, documentation will inform what to ask, while the user's answers will establish their reported experience. A documented expected outcome may be presented as documentation-derived information; it must not silently replace the user's own expectation or observation. Where the two disagree, the disagreement is information to preserve and clarify.

2.7 Evaluation Practices in Prior Research

The reviewed studies evaluate different objects, and their results are not interchangeable. Chaparro et al. assess detection of missing information, while EULER and AstroBR evaluate reproduction-step quality against annotations [3], [12], [16]. Such measures assess an analysis component rather than the whole reporting experience. They do not on their own measure whether a user can complete an interaction successfully.

ChatBR and Acharya and Ginde use measures of generated report quality and similarity to reference content [6], [15]. Reference-based comparison supports reproducible assessment, but a generated completion can resemble a reference without being confirmed for the current incident. ImproBR extends evaluation to executability and reproducibility [7]; this illustrates the value of distinguishing a well-formed report from one whose steps can actually be followed.

BURT combines report-content evaluation with an investigation of user experience [5]. This is directly relevant to an interactive form because users both supply the information and respond to the system's guidance. The additional questions may improve a report while creating additional interaction. Quality and reporting effort should therefore be treated as separate outcomes rather than combined into an unspecified claim of efficiency.

Subjective workload is also distinct from elapsed time. Hart and Staveland's NASA-TLX assesses dimensions including mental demand, temporal demand, effort, frustration, and perceived performance [21]. It provides an established basis for considering perceived workload alongside behavioral measurements. Its inclusion in the review does not establish which instrument or scoring procedure will be adopted; those choices require specification in the evaluation chapter.

Finally, report preparation, understanding a report, reproducing a failure, and repairing a defect are different stages. Repository studies describe relationships between report content and maintenance outcomes [2], [11], but those relationships do not license a causal claim about the proposed interface. An evaluation of report quality and reporting effort can support conclusions about bug reporting. Claims about faster diagnosis or resolution would require measurements of those downstream activities.

2.8 Synthesis, Research Gap, and Research Questions

The literature establishes that incomplete and ambiguous information is a recurring reporting problem. It also provides several existing responses: quality feedback, interactive dialogue, application-informed assessment, LLM-based rewriting, and retrieval-assisted generation. Table 2.1 summarizes selected approaches by their main task and source of context. The categories describe the cited systems rather than an exhaustive taxonomy.

TABLE 2.1
Selected approaches relevant to intelligent bug reporting.

Approach	Main task	Context or evidence
CUEZILLA [1]	Assess reports and recommend additional elements	Report features and developer quality ratings
EULER [12]	Assess reproduction steps and provide feedback	Report text and application interactions
BURT [5]	Guide reporters through an interactive dialogue	Reporter answers and an Android application model
ChatBR [6]	Assess and improve report elements	Report text and generated completions
AstroBR [16]	Assess steps and suggest missing steps	Report text and a GUI execution model
BugMentor [17]	Generate answers to follow-up questions	Current and retrieved historical reports
ImproBR [7]	Improve incomplete or ambiguous sections	Report text and retrieved domain knowledge
Proposed system	Ask clarification questions and structure confirmed answers	Reporter input and service documentation accessed through MCP

The research opportunity is not the invention of interactive reporting or the first use of external knowledge. The reviewed work motivates a more specific empirical question: how useful is a documentation-informed LLM reporting interaction when assessed jointly for report quality, reporting effort, and fidelity to the reporter's evidence? The use of MCP identifies how documentation will be accessed; its presence alone is not evidence of improved reporting.

The intended organizational clients are companies integrating the prototype into support processes for their software services. The reporters are users of those services, and developers and support specialists receive the resulting reports. The

project is independently developed and has no industrial partner; its evaluation will use controlled support scenarios. The planned documentation collection describes the adopting company's service capabilities and previously resolved problems. The intended interaction allows users to write in their own language and receive clarification questions in that language, and produces a structured report for developers. These are project requirements rather than evidence of equivalent performance across languages. The languages included in the evaluation and the provenance of example documentation and resolved cases must be specified in the methodology.

The intended contribution is a prototype and an evaluation of this combination in that service context. The evaluation should separate the effect of interactive assistance from the additional contribution of documentation access. A conventional form provides a reference for reporting practice, while an otherwise comparable assisted form without documentation would help isolate the value of that context. This distinction identifies an attribution issue arising from the review; the experimental protocol is to be specified in Chapter 5.

This synthesis motivates the four research questions stated in Section 1.3.1. RQ1 addresses completeness and clarity, reflecting the distinction between the presence of report elements and their usefulness. RQ2 addresses reporting effort and workload, recognizing that additional guidance may also demand more from the reporter. RQ3 examines fidelity to user-confirmed information, which cannot be established by fluency or structural completeness alone. RQ4 examines the additional effect of documentation, distinguishing it from the effect of LLM-assisted interaction. Together, these questions concern the preparation of a useful and faithful problem description; they do not assume that the system can identify the root cause or accelerate software repair.

Bibliography cited

[1] T. Zimmermann, R. Premraj, N. Bettenburg, S. Just, A. Schröter, and C. Weiss, "What makes a good bug report?" *IEEE Transactions on Software Engineering*, vol. 36, no. 5, pp. 618–643, 2010. DOI: 10.1109/TSE.2010.63 [Online]. Available: https://doi.org/10.1109/TSE.2010.63

[2] M. Soltani, F. Hermans, and T. Bäck, "The significance of bug report elements," *Empirical Software Engineering*, vol. 25, no. 6, pp. 5255–5294, 2020. DOI: 10.1007/s10664-020-09882-z [Online]. Available: https://doi.org/10.1007/s10664-020-09882-z

[3] O. Chaparro et al., "Detecting missing information in bug descriptions," in *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, ACM, 2017, pp. 396–407. DOI: 10.1145/3106237.3106285 [Online]. Available: https://doi.org/10.1145/3106237.3106285

[4] D. Wang, M. Galster, and M. Morales-Trujillo, "Information needs in bug reports for web applications," *Journal of Systems and Software*, vol. 219, 2025, Art. no. 112230. DOI: 10.1016/j.jss.2024.112230 [Online]. Available: https://doi.org/10.1016/j.jss.2024.112230

[5] Y. Song et al., "Toward interactive bug reporting for (Android app) end-users," in *Proceedings of the 30th ACM Joint European Software Engineer-*

ing Conference and Symposium on the Foundations of Software Engineering, ACM, 2022, pp. 344–356. DOI: 10.1145/3540250.3549131 [Online]. Available: https://doi.org/10.1145/3540250.3549131

[6] L. Bo, W. Ji, X. Sun, T. Zhang, X. Wu, and Y. Wei, "ChatBR: Automated assessment and improvement of bug report quality using ChatGPT," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, ACM, 2024, pp. 1472–1483. DOI: 10.1145/3691620.3695518 [Online]. Available: https://doi.org/10.1145/3691620.3695518

[7] E. F. Akyol, M. Dedeler, and E. Tüzün, "ImproBR: Bug report improver using LLMs," Author manuscript; presented at EASE 2026, 2026. DOI: 10.48550/arXiv.2604.26142 arXiv: 2604.26142 [cs.SE]. [Online]. Available: https://arxiv.org/abs/2604.26142

[8] Model Context Protocol. "MCP specification: Resources." Specification revision 2025-06-18, Accessed: Oct. 30, 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2025-06-18/server/resources

[9] Model Context Protocol. "MCP specification: Tools." Specification revision 2025-06-18, Accessed: Oct. 30, 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2025-06-18/server/tools

[10] Z. Ji et al., "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023, Art. no. 248. DOI: 10.1145/3571730 [Online]. Available: https://doi.org/10.1145/3571730

[11] M. Erfani Joorabchi, M. Mirzaaghaei, and A. Mesbah, "Works for me! characterizing non-reproducible bug reports," in *Proceedings of the 11th Working Conference on Mining Software Repositories*, ACM, 2014, pp. 62–71. DOI: 10.1145/2597073.2597098 [Online]. Available: https://doi.org/10.1145/2597073.2597098

[12] O. Chaparro et al., "Assessing the quality of the steps to reproduce in bug reports," in *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ACM, 2019, pp. 86–96. DOI: 10.1145/3338906.3338947 [Online]. Available: https://doi.org/10.1145/3338906.3338947

[13] X. Hou et al., "Large language models for software engineering: A systematic literature review," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 8, pp. 1–79, 2024, Art. no. 220. DOI: 10.1145/3695988 [Online]. Available: https://doi.org/10.1145/3695988

[14] A. McGowan, J. Mahmud, and K. Moran, "Adapting large language models to aid end-users in bug reporting," *Journal of Student-Scientists' Research*, vol. 5, 2023. DOI: 10.13021/jssr2023.3932 [Online]. Available: https://journals.gmu.edu/index.php/jssr/article/view/3932

[15] J. Acharya and G. Ginde, "Can we enhance bug report quality using LLMs?: An empirical study of LLM-based bug report generation," in *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering*, ACM, 2025, pp. 994–1003. DOI: 10.1145/3756681.3756995 [Online]. Available: https://doi.org/10.1145/3756681.3756995

[16] J. Mahmud, A. Saha, O. Chaparro, K. Moran, and A. Marcus, "Combining language and app UI analysis for the automated assessment of bug reproduction steps," in *2025 IEEE/ACM 33rd International Conference on Program Comprehension (ICPC)*, IEEE, 2025, pp. 1–12. DOI: 10.1109/ICPC66645.2025.00010 [Online]. Available: https://doi.org/10.1109/ICPC66645.2025.00010

[17] U. Mukherjee and M. M. Rahman, "BugMentor: Generating answers to follow-up questions from software bug reports using structured information retrieval and neural text generation," *Journal of Systems and Software*, vol. 231, 2026, Art. no. 112636. DOI: 10.1016/j.jss.2025.112636 [Online]. Available: https://doi.org/10.1016/j.jss.2025.112636

[18] Z. Chen, V. Nava-Camal, A. Suleiman, Y. Tang, D. Hou, and W. Shang. "An empirical study on the capability of LLMs in decomposing bug reports." Preprint. arXiv: 2504.20911 [cs.SE]. [Online]. Available: https://arxiv.org/abs/2504.20911 pre-published.

[19] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, Curran Associates, Inc., 2020, pp. 9459–9474. [Online]. Available: https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

[20] N. F. Liu et al., "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024. DOI: 10.1162/tacl_a_00638 [Online]. Available: https://aclanthology.org/2024.tacl-1.9/

[21] S. G. Hart and L. E. Staveland, "Development of NASA-TLX (task load index): Results of empirical and theoretical research," in *Human Mental*

Workload, ser. Advances in Psychology, P. A. Hancock and N. Meshkati, Eds., vol. 52, North-Holland, 1988, pp. 139–183. DOI: 10.1016/S0166-4115(08)62386-9 [Online]. Available: https://doi.org/10.1016/S0166-4115(08)62386-9